

Documentación Paso a Paso del Proyecto Poké-Terminal

Luis Enrique Cortijo Gonzales

July 16, 2025

1 Objetivo

Desarrollar una terminal personalizada en Linux que muestre un sprite de Pokémon como fondo estático al iniciar la terminal. A diferencia del repositorio original (que imprime arte ASCII), este proyecto utiliza sprites gráficos reales en formato SIXEL, permaneciendo visibles tras ejecutar comandos o usar el comando `clear`.

2 Requisitos Técnicos

- Terminal compatible con gráficos SIXEL, como `mlterm`.
- Herramienta `pokeshell` para seleccionar y mostrar sprites de Pokémon.
- Herramienta `img2sixel` (proporcionada por el paquete `libsixel-bin`) para gestionar el renderizado de imágenes.
- Personalización automática mediante el archivo de configuración `.bashrc`.

3 Proceso General

3.1 Instalación de Herramientas

Se instalan todas las dependencias necesarias mediante el gestor de paquetes del sistema. El repositorio `pokeshell` se instala usando `just`, garantizando que esté disponible como comando global.

3.2 Configuración de `mlterm` como Terminal Predeterminada

Se configura `mlterm` como el terminal por defecto en el sistema utilizando el mecanismo estándar de `update-alternatives` en Debian/Ubuntu (mejor sistema operativo es fedora).

3.3 Personalización Visual

Se modifica el archivo `.bashrc` del usuario, agregando un bloque personalizado que:

- Cambia el color de fondo a negro y el texto a blanco mediante secuencias OSC.
- Genera un sprite Pokémon aleatorio usando `pokeshell` una sola vez por sesión.
- Convierte el sprite a formato SIXEL mediante `img2sixel` y lo guarda en un archivo temporal.
- Dibuja el sprite al iniciar la terminal y después de ejecutar el comando `clear`.
- Define el comando `pokerefresh` para cambiar manualmente el Pokémon.

4 Explicación Paso a Paso y Contexto en Sistemas Operativos

El desarrollo de este proyecto consistió en los siguientes pasos secuenciales:

1. **Análisis del entorno de terminal:** Se identificó la necesidad de una terminal capaz de renderizar imágenes. Se eligió `mlterm` por su soporte nativo de SIXEL.
2. **Elección de herramienta gráfica:** Se adoptó `pokeshell` como generador de sprites Pokémon y `img2sixel` para redimensionarlos y procesarlos.
3. **Definición del comportamiento persistente:** Se diseñó un método para almacenar el sprite en un archivo temporal y redibujarlo cada vez que se ejecuta `clear`, manteniendo la imagen como un fondo persistente.
4. **Integración mediante el shell:** Mediante la modificación del archivo `.bashrc`, se establecieron alias y funciones que aseguran la visualización automática y continua del sprite.
5. **Automatización del proceso:** Para facilitar su despliegue en diferentes sistemas, se creó un script integral que realiza toda la instalación y configuración de forma autónoma.

En el contexto de sistemas operativos, este proyecto involucra aspectos como:

- Configuración de terminales virtuales y personalización del entorno de usuario.
- Manipulación del shell de usuario (`bash`) mediante scripts.

- Gestión de procesos (al usar comandos y scripts ejecutables).
- Uso de dispositivos de salida estándar y secuencias de control ANSI para personalización visual.
- Aprovechamiento de capacidades gráficas específicas (SIXEL) en terminales compatibles.

Este proyecto representa una práctica concreta de personalización del entorno de usuario a nivel terminal, combinando programación de scripts, gestión de procesos y configuración de shell, temas fundamentales en la administración y manejo de sistemas operativos.

5 Comentario final

En este proyecto las principales dificultades fueron el elegir la terminal adecuada, en este caso el `mlterm` ya que GNOME no soportaba imágenes en el fondo. Posteriormente fue hacerla la terminal principal y la modificación del `bash` junto al componente aleatorio del código modificado de `pokeshell` para que cada que se abra `mlterm` se genere en el fondo un nuevo sprite de `pokemon` persistente. Hay espacio de mejora como crear una `syscall` que genera los `randoms` o elegir un nuevo `pokemon` o poder setear mejor el tamaño de estos. Pero para la entrega es funcional y creo que podría ir mejorando en el futuro.